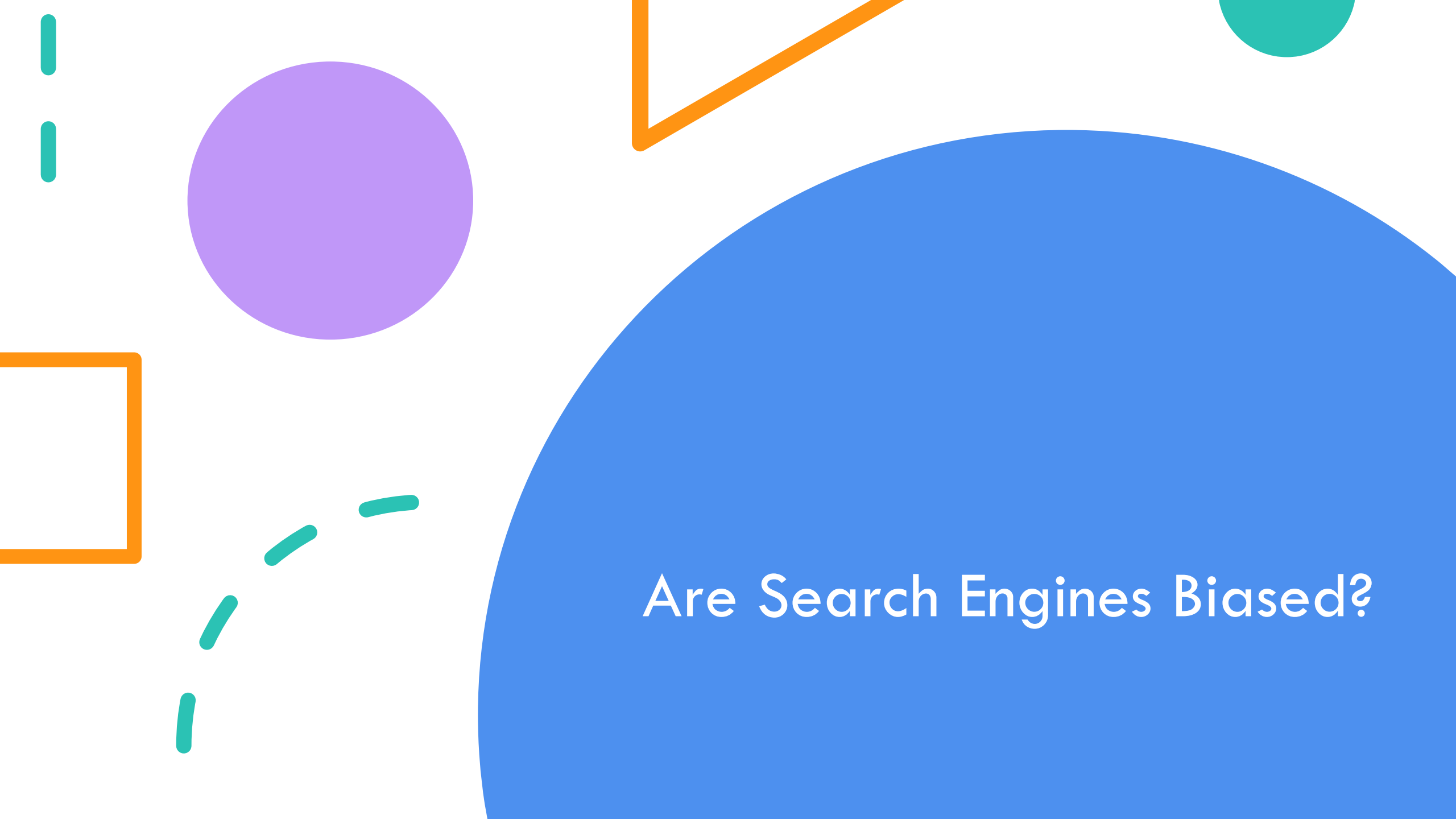




Are Search Engines Biased?

کتابخانه‌ها

```
1 import sqlite3
2 import requests
3 from bs4 import BeautifulSoup
4 from scipy.stats import chi2_contingency
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 import tkinter as tk
8 from tkinter import ttk
9 import numpy as np
10
```

• **Sqlite3**: برای عملیات پایگاه داده.

• **Requests**: برای ارسال درخواست‌های
.HTTP

• **BeautifulSoup**: برای تجزیه محتوای
.HTML

• **chi2_contingency** از **scipy.stats**:
برای انجام آزمون‌های مربع کای.

• **matplotlib.pyplot** و **seaborn**: برای
تجسم داده‌ها.

• **tkinter** و **ttk**: برای ایجاد برنامه‌های **GUI**.

• **Numpy**: برای عملیات عددی.

ایجاد و اتصال به پایگاه داده

```
12 |
13 | # Initialize SQLite database
14 | usage
15 | def create_database():
16 |     conn = sqlite3.connect('search_history.db')
17 |     cursor = conn.cursor()
18 |     cursor.execute('''CREATE TABLE IF NOT EXISTS search_results (
19 |         id INTEGER PRIMARY KEY AUTOINCREMENT,
20 |         search_query TEXT,
21 |         google_results TEXT,
22 |         bing_results TEXT,
23 |         bias INTEGER,
24 |         search_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP)''')
25 |
26 |     conn.commit()
27 |     conn.close()
28 |
29 |
30 | create_database()
31 |
```

پایگاه داده ایجاد شده یا به آن متصل می‌شود و یک جدول برای ذخیره نتایج جستجو ایجاد می‌شود.

تابع برای دریافت نتایج جستجو از گوگل

```
31
32 # Google Custom Search API configuration
33 api_key = 'AIzaSyB_QW66IYb6X-1GdvobQio3qnMpbVlipJY'
34 cx = '661a5f6476044440d'
35
36
37 # Function to fetch search results from Google
38 usage
39 def google_search(query, api_key, cx, num_results=10):
40     results = []
41     start = 1
42     while len(results) < num_results:
43         url = f"https://www.googleapis.com/customsearch/v1?q={query}&key={api_key}&cx={cx}&start={start}"
44         response = requests.get(url)
45         response_json = response.json()
46         items = response_json.get('items', [])
47         results.extend([item['link'] for item in items])
48         start += 10
49
50         if not items or len(items) < 10:
51             break # No more results available
52
53     return results[:num_results]
```

تابع برای دریافت نتایج جستجو از بینگ

```
56 # Function to fetch search results from Bing
57 1 usage
58 def bing_search(query, num_results=10):
59     try:
60         url = f'https://www.bing.com/search?q={query}'
61         headers = {'User-Agent': 'Mozilla/5.0'}
62         response = requests.get(url, headers=headers)
63         soup = BeautifulSoup(response.text, features='html.parser')
64         results = []
65         for b in soup.find_all(name='li', class_='b_algo'):
66             anchors = b.find_all('a')
67             if anchors:
68                 link = anchors[0]['href']
69                 results.append(link)
70                 if len(results) >= num_results:
71                     break # Stop after collecting the desired number of results
72             return results
73 except Exception as e:
74     print(f"Error in Bing search: {e}")
75     return []
```

مقایسه نتایج جستجو

```
77 # Function to compare Google and Bing search results
    2 usages
78 def compare_results(results1, results2):
79     common_results = set(results1).intersection(results2)
80     return len(common_results), common_results
81
```

این تابع لینک‌های مشترک بین دو مجموعه نتایج جستجو را شناسایی می‌کند.

تابع برای تحلیل تعصب با استفاده از آزمون مربع کای

```
84 def analyze_bias(results1, results2):  
85     common_count, common_links = compare_results(results1, results2)  
86     obs1 = len(results1) + 0.5  
87     obs2 = len(results2) + 0.5  
88     contingency_table = [[obs1, obs2], [obs1 - common_count, obs2 - common_count]]  
89     chi2, p, dof, expected = chi2_contingency(contingency_table)  
90     return chi2, p  
91
```

این تابع یک جدول تلافی بر اساس تعداد نتایج ایجاد می‌کند و آزمون مربع کای را برای تشخیص تعصب انجام می‌دهد.

تابع برای تجسم مقایسه نتایج

```
84 def analyze_bias(results1, results2):  
85     common_count, common_links = compare_results(results1, results2)  
86     obs1 = len(results1) + 0.5  
87     obs2 = len(results2) + 0.5  
88     contingency_table = [[obs1, obs2], [obs1 - common_count, obs2 - common_count]]  
89     chi2, p, dof, expected = chi2_contingency(contingency_table)  
90     return chi2, p  
91
```

این تابع یک نمودار میله‌ای ایجاد می‌کند که تعداد نتایج از گوگل و بینگ را مقایسه می‌کند.

توابع برای انجام آزمون Dixon Q

```
def dixon_q_test(data, significance_level=0.05):  
    n = len(data)  
    if n < 3:  
        return None, None # Not enough data to perform the test  
  
    data_sorted = sorted(data)  
    q_statistic = (data_sorted[-1] - data_sorted[-2]) / (data_sorted[-1] - data_sorted[0])  
  
    q_critical_values = {  
        0.1: [0.941, 0.765, 0.642, 0.560, 0.507, 0.468, 0.437, 0.412, 0.392, 0.376, 0.361, 0.349, 0.338, 0.329, 0.320],  
        0.05: [0.978, 0.829, 0.710, 0.625, 0.568, 0.526, 0.493, 0.466, 0.444, 0.426, 0.410, 0.396, 0.384, 0.374, 0.364],  
        0.02: [0.994, 0.926, 0.821, 0.740, 0.688, 0.634, 0.598, 0.568, 0.544, 0.523, 0.507, 0.493, 0.480, 0.468, 0.457],  
        0.01: [0.999, 0.978, 0.889, 0.829, 0.788, 0.741, 0.707, 0.679, 0.654, 0.634, 0.617, 0.601, 0.588, 0.575, 0.564]  
    }  
  
    if significance_level not in q_critical_values:  
        raise ValueError("Significance level must be one of: 0.1, 0.05, 0.02, 0.01")  
  
    q_critical = q_critical_values[significance_level][n - 3]  
  
    return q_statistic, q_statistic > q_critical
```

تابع برای گرفتن کوئری از کاربر

```
142 def on_search():
143     query = entry1.get()
144     global google_results, bing_results
145     google_results = google_search(query, api_key, cx)
146     bing_results = bing_search(query)
147
148     google_result_text.delete(indent=1.0, tk.END)
149     bing_result_text.delete(indent=1.0, tk.END)
150
151     google_result_text.insert(tk.END, "\n".join(google_results))
152     bing_result_text.insert(tk.END, "\n".join(bing_results))
153
154     chi2, p_value = analyze_bias(google_results, bing_results)
155     q_statistic, is_outlier = analyze_bias_with_dixon(google_results, bing_results)
156
157     if q_statistic is None or is_outlier is None:
158         result_text.set(
159             f"Chi-square: {chi2:.2f}, p-value: {p_value:.2e}\nDixon Q: N/A, Outlier: N/A")
160     else:
161         result_text.set(
162             f"Chi-square: {chi2:.2f}, p-value: {p_value:.2e}\nDixon Q: {q_statistic:.2f}, Outlier: {is_outlier}")
163
164     bias = int(p_value < 0.05 or (is_outlier is not None and is_outlier))
165
166     conn = sqlite3.connect('search_history.db')
167     cursor = conn.cursor()
168     cursor.execute(_sql 'INSERT INTO search_results (search_query, google_results, bing_results, bias) VALUES (?, ?, ?, ?)',
169                   __parameters__ (query, "\n".join(google_results), "\n".join(bing_results), bias))
170     conn.commit()
```

تابع برای تجسم پیشرفته توزیع نتایج جستجو

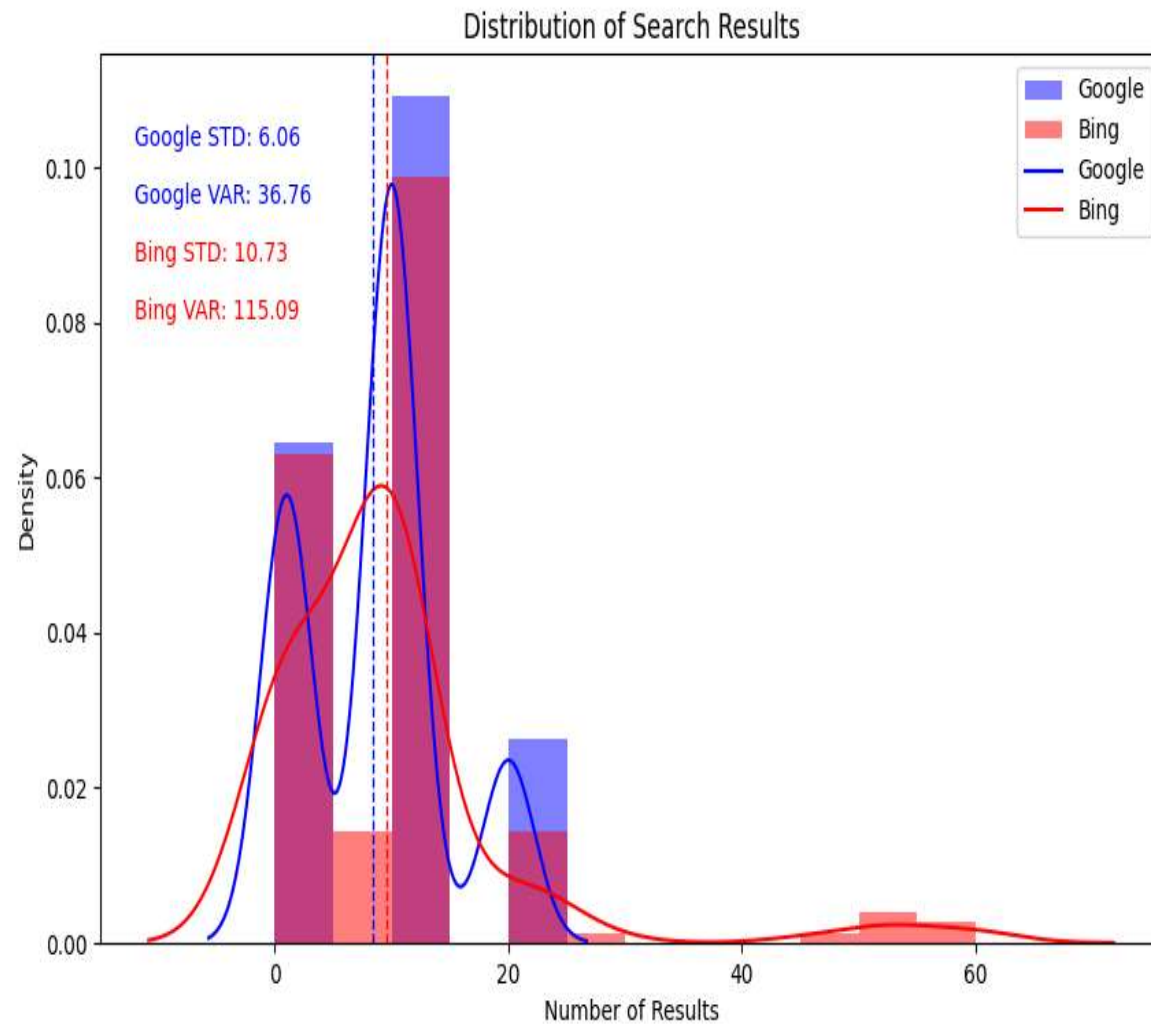
```
1 usage
232 def on_compare_results():
233     google_counts, bing_counts = calculate_statistics()
234
235     plt.figure(figsize=(10, 6))
236     plt.hist(google_counts, bins=range(0, 61, 5), alpha=0.5, label='Google', color='b', density=True)
237     plt.hist(bing_counts, bins=range(0, 61, 5), alpha=0.5, label='Bing', color='r', density=True)
238     sns.kdeplot(google_counts, color='b', label='Google')
239     sns.kdeplot(bing_counts, color='r', label='Bing')
240     plt.axvline(np.mean(google_counts), color='b', linestyle='dashed', linewidth=1)
241     plt.axvline(np.mean(bing_counts), color='r', linestyle='dashed', linewidth=1)
242
243     google_var = np.var(google_counts)
244     google_std = np.std(google_counts)
245     bing_var = np.var(bing_counts)
246     bing_std = np.std(bing_counts)
247
248     plt.text(x=40, y=0.25, s=f'Google STD: {google_std:.2f}\nGoogle VAR: {google_var:.2f}', color='blue')
249     plt.text(x=40, y=0.2, s=f'Bing STD: {bing_std:.2f}\nBing VAR: {bing_var:.2f}', color='red')
250
251     plt.title('Distribution of Search Results')
252     plt.xlabel('Number of Results')
253     plt.ylabel('Density')
254     plt.legend()
255     plt.figtext(x=0.15, y=0.8, s=f'Google STD: {google_std:.2f}', color='blue')
256     plt.figtext(x=0.15, y=0.75, s=f'Google VAR: {google_var:.2f}', color='blue')
257     plt.figtext(x=0.15, y=0.7, s=f'Bing STD: {bing_std:.2f}', color='red')
258     plt.figtext(x=0.15, y=0.65, s=f'Bing VAR: {bing_var:.2f}', color='red')
259     plt.show()
```


رابط کاربری گرافیکی با استفاده از Tkinter

```
294 result_text = tk.StringVar()
295 result_label = ttk.Label(frame1, textvariable=result_text)
296 result_label.grid(row=5, column=8, columnspan=3, padx=5, pady=5)
297
298 google_result_label = ttk.Label(frame1, text="Google Results:")
299 google_result_label.grid(row=1, column=8, padx=5, pady=5, sticky=tk.W)
300
301 google_result_text = tk.Text(frame1, height=10, width=50)
302 google_result_text.grid(row=2, column=8, columnspan=3, padx=5, pady=5)
303
304 bing_result_label = ttk.Label(frame1, text="Bing Results:")
305 bing_result_label.grid(row=3, column=8, padx=5, pady=5, sticky=tk.W)
306
307 bing_result_text = tk.Text(frame1, height=10, width=50)
308 bing_result_text.grid(row=4, column=8, columnspan=3, padx=5, pady=5)
309
310 frame2 = ttk.Frame(win1)
311 frame2.grid(row=1, column=9, sticky="N")
312
313 bias_analysis_button = ttk.Button(frame2, text="Bias Analysis of Fast Results", command=on_bias_analysis)
314 bias_analysis_button.grid(row=0, column=0, padx=10, pady=10)
315
316 comparison_button = ttk.Button(frame2, text="Comparison of Search Results", command=on_compare_results)
317 comparison_button.grid(row=0, column=1, padx=10, pady=10)
318
319 history_button = ttk.Button(frame2, text="Search History", command=show_history)
320 history_button.grid(row=0, column=2, padx=10, pady=10)
321
322
323 win1.mainloop()
324
```

یک پنجره با فیلدهای ورودی برای جستجو و یک دکمه برای شروع جستجو ایجاد می‌شود. زمانی که دکمه **"Search"** کلیک می‌شود، نتایج از گوگل و بینگ بازیابی، نمایش و تجسم می‌شوند.

تفسیر کامل نمودار و بررسی تعصب الگوریتم‌های جستجو



تفسیر کامل نمودار و بررسی تعصب الگوریتم‌های جستجو

1. تفسیر نمودار نموداری که مشاهده می‌کنید، توزیع نتایج جستجو از دو موتور جستجوی گوگل و بینگ را نشان می‌دهد. در این نمودار:

- میله‌های آبی مربوط به نتایج گوگل است.
- میله‌های قرمز مربوط به نتایج بینگ است.
- خطوط پیوسته آبی و قرمز نشان‌دهنده نمودارهای KDE برای گوگل و بینگ هستند.
- میله‌های شفاف آبی و قرمز نیز نشان‌دهنده هیستوگرام نتایج جستجو هستند.

2. محاسبات آماری در این نمودار، مقادیر زیر نمایش داده شده است:

انحراف معیار و واریانس برای هر دو موتور جستجو:

- گوگل: انحراف معیار **6.06** و واریانس **36.76**
- بینگ: انحراف معیار **10.73** و واریانس **115.09**

آیا الگوریتم جستجو تعصب دارد؟

با توجه به انحراف معیار و واریانس بالا برای نتایج بینگ، می‌توان نتیجه گرفت که نتایج بینگ پراکندگی بیشتری دارند و به طور کلی تعصب خاصی در نتایج هر دو موتور جستجو دیده نمی‌شود.

اما برای نتیجه‌گیری دقیق‌تر، می‌توان به نتایج آزمون مربع کای و آزمون **Dixon Q** مراجعه کرد. در این کد، آزمون‌های آماری برای بررسی بایاس در نتایج جستجو به کار گرفته شده‌اند:

آزمون مربع کای برای بررسی استقلال نتایج جستجوی گوگل و بینگ.

آزمون **Dixon Q** برای شناسایی موارد پرت در نتایج جستجو.

با توجه به نمودار و نتایج آماری، پراکندگی بیشتر در نتایج بینگ مشاهده می‌شود، اما این لزوماً به معنای تعصب بودن الگوریتم جستجوی بینگ نیست. بلکه نشان‌دهنده‌ی تنوع بیشتر نتایج بینگ است. برای تشخیص دقیق تعصب، باید به نتایج آزمون‌های آماری و تحلیل‌های بیشتر نیز توجه کرد.